CP 312, Fall 2025
Assignment 4

1. [10 marks] **Dynamic Programming.** The *longest exponential subsequence* of a sequence of n positive integers $a_1, a_2, \ldots, a_n$ is the longest subsequence such that each integer in the subsequence is at least twice as large as the previous one.

   For example, given the instance
   `1, 256, 16, 4096, 4, 1024, 64, 16384, 2, 128, 32, 8192, 8, 2048, 512, 32768`
   one of the longest exponential subsequences is `1, 16, 64, 128, 2048, 32768`.
   (Note that there are multiple exponential subsequences of length 6 for this example.) Design an algorithm that finds a longest exponential subsequence in time $O(n^2)$:

   1. Give a precise definition of a subproblem that can be used to solve your problem with a dynamic programming algorithm.
   2. Describe and justify the recurrence rule that will be used to compute the solutions to the subproblems defined in part (a).
   3. Provide a pseudocode for your dynamic programming algorithm that solves your problem using the subproblems and recurrence rule defined above. Your algorithm needs to find a longest exponential subsequence: the answer is length of the longest exponential subsequence and also list of elements forming it. For the input above possible output is:

      ```
      length: 6
      subsequence: 1, 16, 64, 128, 2048, 32768
      ```

   4. Analyze the time complexity of the algorithm obtained in part (c).

(a) Definition

Let `L[i]` denote the length of the longest exponential sequence ending at position `i` with element `a[i]`.

(b) Recurrence Rule

`Let L[i] = 1 + max{L[j] : 1 <= j <= i and a[i] >= 2 * a[j]}`

If no such `j` exists then `L[i] = 1`.

(c) Psuedocode

```
LongestExponentialSubsequence():
    // Intialized arrays
    L[1 ...n] <- array of lenngth n, all elements = 1
    prev[1 ...n] <- array of lenngth n, all elements = -1

    // Fill dp table
    for i = 2 to n do
        for j = 1 to i - 1 do
            if a[i] >= 2 * a[j] then
                if L[j] + 1 >= L[i] then
                    L[i] = L[j] + 1
                    prev[i] = j

    // Find position of the max length
    maxLen = 0
    maxIndex = -1
    for i = 1 to n do
        if L[i] > mazLen then
            maxLen = L[i]
            maxIndex = i

    // Reconstruct the subsequence
    subsequence = empty list
    current = maxIndex
    while current != -1 do
        prepend to a[current] to subsequence
        current = prev[current]

    return (maxLen, subsequence)
```

Output Format:

```
Length: maxLen
subsequence: elements in subsequence
```

(d) Time Complexity Analysis

Analysis:

- The outer loop runs `n` times (from `i = 2` to `n`)
- The inner loop runs at most `i - 1` times for each `i`
- Total Comparisons: `1 + 2 + 3 + ... + n - 1 = O(n^2)`
- Find max length: `O(n)`
- Reconstructing the subsequence: `O(n)` is the worst case

Total Time Complexity: O(n^2) Space Complexity: O(n) for storing the `L` and `prev` arrays.

---

2. [7 marks] **Dynamic Programming.** Consider a variation of the Knapsack problem. There are two knapsacks that have capacity $W_1 > 0$ and $W_2 > 0$, respectively. There are n items $1, 2, \ldots, n$. Item i has weight $w(i) > 0$ and two values $v_1(i) > 0$ and $v_2(i) > 0$. Here $v_k(i)$ is the value one gains by putting item i into knapsack k ($k = 1, 2$). Note, that values $v_1(i)$ and $v_2(i)$ are generally speaking different. The "Two Knapsacks Problem" is to find two *disjoint* subsets of items S_1 and S_2, such that

   1. $\sum_{i \in S_1} w(i) \leq W_1$,
   2. $\sum_{i \in S_2} w(i) \leq W_2$, and
   3. $V = \sum_{i \in S_1} v_1(i) + \sum_{i \in S_2} v_2(i)$ is maximized.

   Give a dynamic programming algorithm to find the maximum value V. Your algorithm does not need to find the sets S_1 and S_2. Clearly indicate what your subproblems are, and the order in which you solve them. Justify correctness of your algorithm, and analyze its running time. Is your algorithm a polynomial-time algorithm? Why or why not?

Subproblem Definition

Subproblem: `DP[i][w1][w2]` = max total achievable value using `1` to `i`, where the total weight of th eknapsak 1 is at most `w1` and the total weight of knapsak 2 is at most `w2`.

Recurrence Rule

For each item `i`, we all have 3 choices:

1. Place it in knapsak 1 (if it fits)
2. Place it in knapsak 2 (if it fits)
3. Don't include it in either knapsak

```
DP{
    DP[i - 1][w1][w2], // Don't take item i
    DP[i - 1][w1 - w(i)][w2] + v1(i)    if w(i) <= w1, // Put in knapsak 1
    DP[i - 1][w1][w2 - w(i)] + v2(i)    if w(i) <= w2 // Put in knapsak 2
}
```

Base case: `DP[0][w1][w2] = 0` for all valid `w1` and `w2` (no items means zero value).

Algorithm

```
TwoKnapsaks(n, w[], W1, W2, v1[], v2[]):
    // Initialize DP table
    DP[0 ...n][0 ...W2][0 ...W2] <- all elements = 0

    // Fill DP table
    for i = 1 to n do
        for w1 = 0 to W1 do
            for w2 = 0 to W2 do
                // option 1: Don't take item 1
                DP[i][w1][w2] = DP[i - 1][w1][w2]

                // Option 2: Put item i in knapsak 1
                if w(i) <= w1 then
                    DP[i][w1][w2] = max(DP[i][w1][w2], DP[i - 1][w1 - w(i)][w2] + v1(i)

                // Option 3: Put item i in knapsak 2
                if w(i) <= w2 then
                    DP[i][w1][w2] = max(DP[i][w1][w2], DP[i - 1][w1][w2 - w(i)] + v1(i)

    return DP[i][w1][w2]
```

Order of Solving the Subproblem

The subproblem is solved in order of of increasing item index `i`, for each `i`, iterate through all possible weight combinations `(w1, w2)`.

Correctness Justification

The algorithm is correct because:

1. All posssible placements are considered for items (knapsak 1, knapsak 2, or neither).
2. Each subproblem `DP[i][w1][w2]` depends only on subproblems with fewer items (`i - 1`).
3. The optimal solution for `i` is built from optimal solution for `i - 1` items.
4. The recurrence correctly captures the constraint that items must be disjoint between knapsaks

Time Complexity Analysis

- 3 nested loops: `i` iterate from 1 to `i`, `w1` from 0 to `w2`, `w2` from 0 to `w2`
- Each iteration performs constant work
- **Time Complexity: O(n * w1 * w2)**
- **Space Complexity: O(n * w1 * w2)** but can be optimized to O(w1 * w2) by rolling array

Polynomial Time Analysis

This is Not a Polynomial-Time algorithm in a strict sense because:

- The running time is polynomial in `w1`, `w2` and `n`
- However, `w1` and `w2` are numerical values, not an input size
- The input size representing `W1` is `O(log W1)` bits
- The algorithm is **psuedo-polynomial** because the input size is not polynomial but the numerial value of the input is
- This is a weakly NP-Hard problem

---

3. [7 marks] The *distance* between two vertices in a connected undirected graph is the length of the shortest simple path between them (number of edges on the shortest simple path). The *diameter* of a connected undirected graph is the maximum distance between vertices in the graph. Give an efficient algorithm to compute the diameter of a graph. Provide justification of the correctness and a its running time complexity in terms of $|V|$ and $|E|$.

Algorithm

The diameter is the maximum distance bewteen any pair of verticies. We can then compute it by running BFS from every vertex and finding the max distance encountered.

```
GraphDaameter(G = (V, E)):
    diameter = 0

    for each vertex u in V do
        // Run BFS from u
        dist[v] = inf for all v in V
        dist[u] = 0
        queue = empty queue
        enqueue(queue, 0)
        maxDist = 0

        while queue is not empty do
            v = dequeue(queue)
            for each neighbor w of v do
                if dist[w] = inf then
                    dist[w] = dist[v] + 1
                    maxDist = max(maxDist, dist[w])
                    enqueue(queue, ws)

        diameter = max(diameter, maxDist)

    return diameter
```

Correctness Justification

Correctness:

1. BFS from a vertex `u` corrrectly computes the shortest path from `u` to all reachable verticies
2. The max distance found during the BFS from `u` is the farthest from `u`
3. By running a BFS from every vertex, we can find the max distance between any pair of verticies
4. Since the graph is connected, every vertex would be reachable from every other vertex
5. The diameter is correctly computed as the max over all the distances

Time complexity

- BFS from a single vertex takes `O(|V| + |E|)` time
- We run BFS all `|V|` vertices
- **Total Time Complexity: O(|V| * (|V| + |E|)) = O(|V|^2 * |E|)**

For Sparse graphs where `|E| = O(|V|)`, this is `O(|V|^2)`. For dense graphs where `|E| = O(|V|^2)`, this is `O(|V|^3)`.

Space Complexity: $\mathbf{O(|V| + |E|)}$ for sorting the graph and auxiliary arrays.

---

4. [6 marks] Solve previous problem in simplified settings: given graph is a binary tree. The *diameter* of a tree is the maximum distance between vertices in the tree. Solve this problem using divide and conquer approach. Your algorithm takes a pointer to the root of a binary tree as a parameter, and returns the diameter.

   Is this algorithm asymptotically faster than algorithm in question 3?

   Algorithm

   For the binary search tree, the diameter can be computed recursively. For each node, the diameter ether:

   1. Passes through the root (i.e. the longest path from the left subtree through the root to the right subtree)
   2. Is entirely in the left subtree
   3. Is entirely in the right subtree

```
TreeDiameter(root):
    if root = NULL then
        return 0

    result = DiameterAndHeight(root)
    return result.diameter

DiameterAndHeight(root):
    // returns a structure containing diameter and height
    if node = NULL then
        return {diameter: 0, height: 0}

    // Recursively  compute for the left and right subtrees
    left = DiameterAndHeight(node.left)
    right = DiameterAndHeight(node.right)

    // height of the current node
    height = 1 + max(left.height, right.height)

    // Diameter is the max of:
    // 1. diameter of left subtree
    // 2. diameter of right subtree
    // 3. longest path through current nod (left.height + right.height)
    diameter = max(left.diameter, right.diameter, left.height + right.height)
    return {diameter: diameter, height: height}
```

Correctness Justficiation

Correctness:

1. **Base case:** A empty tree has a diameter of 0
2. **Recursive case:** For any node, the diameter is either:
   - In the left subtree (handled by recrusive call)
   - In the right subtree (handled by recrusive call)
   - Passes through the current node (computed as sum of heights)
3. The height of each subtree gives us the longest path from the subtree to its root
4. Adding the heights of the left and right subtrees gives the longest path through the current node
5. We then take the max of all 3 possiblilites

Time Complexity

- Each node is visited exactly once
- At each node, we perform operations under a time constraint
- **Time Complexity: O(n)** where `n = |V|` is the number of nodes in the tree

Space complexity: O(h) where `h` is the height of the tree due to the recursive stack

- Best case (Balanced tree): `O(log n)`
- Worst case (skewed tree): `O(n)`

Question 3 comparison

Yes, the question 4 algorithm is asymtotically faster than the algorithm in question 3.

- Question 3 (general graph): `O(|V|^2 + |V| * |E|)`
- Question 4 (binary tree): `O(n)` where `n = |V|` For a tree, `|E| = |V| - 1 = O(|V|)`, so the algorithm from question 3 would be `O(|V|^2)` for trees.

The divide-and-conquer approach is faster because:

1. It exploits the structure of the tree (0 cycles)
2. Each node is visited once instead of being the source of a BFS
3. The diameter is then computed in a single traversal using a recursive structure

Speedup Factor: O(|V|) vs O(|V|^2) Thus it is a faster by a factor of `|V|`.